

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:25

Annexure A

Coding challenge

Code of Conduct:

*I HARISH KUMAR G 192521340 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A - Coding Challenge

1. Write a Python script that generates a star topology diagram (ASCII or graphical) given n hosts and a central switch. The script must print each host label, the switch name, and the connecting Cat6 UTP cable length.

```
therla bhai.py - C:\Users\DELL\AppData\Local\Programs\Python\Python313\therla bhai.py (...
File Edit Format Run Options Window Help
import random
import time

def simulate_csma_cd():
    hosts = ["Host-A", "Host-B", "Host-C", "Host-D"]
    log_filename = "csma_log.txt"

    print(f"\n--- Simulating CSMA/CD for {len(hosts)} hosts ---")

    with open(log_filename, "w") as log_file:
        log_file.write("CSMA/CD Simulation Log\n" + "="*30 + "\n")

        for transmission_round in range(1, 4):
            print(f"\nTransmission Round {transmission_round}:")
            log_file.write(f"\nRound {transmission_round}:\n")

            active_transmitters = random.sample(hosts, k=random.randint(2, 4))
            print(f"Active transmitters trying to send: {active_transmitters}")

            if len(active_transmitters) > 1:
                print(">>> COLLISION DETECTED! Multiple frames on wire.")
                log_file.write(f"COLLISION: Active hosts {active_transmitters} c")

                for host in active_transmitters:
                    backoff_slots = random.randint(1, 4)
                    backoff_time = backoff_slots * 51.2 # bit
                    log_file.write(f" -> {host} back-off interval: {backoff_tim")
                    print(f"    * {host} backs off for {backoff_time} us")
            else:
                msg = f"Transmission successful for {active_transmitters[0]} wit
                print(msg)
                log_file.write(f"SUCCESS: {msg}\n")

            print(f"\nCSMA/CD events successfully logged to '{log_filename}'.")

    simulate_csma_cd()
```

```
IDLE Shell 3.13.7
Python 3.13.7 (tags/v3.13.7:bce1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit
AMD64] on win32
Enter "help" below or click "Help" above for more information.

>>>
= RESTART: C:\Users\DELL\AppData\Local\Programs\Python\Python313\therla bhai.py
--- Simulating CSMA/CD for 4 hosts ---

Transmission Round 1:
Active transmitters trying to send: ['Host-A', 'Host-B']
>>> COLLISION DETECTED! Multiple frames on wire.
    * Host-A backs off for 102.4 us
    * Host-B backs off for 153.60000000000002 us

Transmission Round 2:
Active transmitters trying to send: ['Host-B', 'Host-D', 'Host-C', 'Host-A']
>>> COLLISION DETECTED! Multiple frames on wire.
    * Host-B backs off for 51.2 us
    * Host-D backs off for 102.4 us
    * Host-C backs off for 51.2 us
    * Host-A backs off for 153.60000000000002 us

Transmission Round 3:
Active transmitters trying to send: ['Host-C', 'Host-A']
>>> COLLISION DETECTED! Multiple frames on wire.
    * Host-C backs off for 102.4 us
    * Host-A backs off for 204.8 us

CSMA/CD events successfully logged to 'csma_log.txt'.
>>>
```

2. Code a function `validate_segment(length_m)` that returns `True` if the UTP segment is within IEEE 802.3 limits (≤ 100 m) and `False` with an error message otherwise. Include at least 3 test cases.

```
main.py Visualize Run Output
1 def validate_segment(length_m):
2
3     MAX_LIMIT = 100.0
4     if 0 < length_m <= MAX_LIMIT:
5         return True, f"Segment length {length_m}m is valid (Within IEEE 802.3 limits)."
6     elif length_m > MAX_LIMIT:
7         return False, f"Error: Segment length {length_m}m exceeds the 100m IEEE 802.3 limit!"
8     else:
9         return False, f"Error: Invalid length {length_m}m. Must be greater than 0."
10 print("\n--- UTP Validation Test Cases ---")
11 test_lengths = [45.5, 100.0, 115.2]
12 for length in test_lengths:
13     is_valid, message = validate_segment(length)
14     print(f"Testing {length}m -> Valid: {is_valid} | Message: {message}")
```

```
ERROR!

--- UTP Validation Test Cases ---
Testing 45.5m -> Valid: True | Message:
Segment length 45.5m is valid (Within IEEE
802.3 limits).
Testing 100.0m -> Valid: True | Message:
Segment length 100.0m is valid (Within IEE
802.3 limits).
Testing 115.2m -> Valid: False | Message:
Error: Segment length 115.2m exceeds the 1
IEEE 802.3 limit!

=== Code Execution Successful ===
```

3. Write a script that simulates CSMA/CD on a star topology with 4 hosts. Log every collision detection event and the back-off interval to a file named `csma_log.txt`.

```

import socket
import threading
import time

mac_table = {}

def switch_handler(server_socket):
    server_socket.settimeout(2.0)
    try:
        conn, addr = server_socket.accept()
        data = conn.recv(1024).decode()
        if data:
            parts = data.split("|")
            src_mac, dest_mac, message = parts[0], parts[1], parts[2]

            mac_table[src_mac] = addr

            print(f"\n[Switch Frame Processed]")
            print(f"  Source MAC: {src_mac} | Dest MAC: {dest_mac}")
            print(f"  Payload: '{message}'")
            print(f"  Updated Switch MAC Table: {mac_table}")

            conn.sendall(b"Frame Forwarded Successfully")
            conn.close()
    except socket.timeout:
        pass

def run_socket_simulation():
    print("\n--- Socket & MAC Forwarding Simulation ---")
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind(('127.0.0.1', 0)) #
    server.listen(1)
    port = server.getsockname()[1]

    t = threading.Thread(target=switch_handler, args=(server,))
    t.start()

    # Client Transmission
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client.connect(('127.0.0.1', port))

    frame_data = "AA:BB:CC:00:11:22|11:22:33:44:55:66|Hello Switch!"
    client.sendall(frame_data.encode())
    response = client.recv(1024).decode()
    print(f"Client received response: {response}")

    client.close()
    t.join()
    server.close()

```

Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit AMD64] on win32
Enter "help" below or click "Help" above for more information.

```

>>>
= RESTART: C:\Users\DELL\AppData\Local\Programs\Python\Python313\therla bhail.py

--- Socket & MAC Forwarding Simulation ---

[Switch Frame Processed]
  Source MAC: AA:BB:CC:00:11:22 | Dest MAC: 11:22:33:44:55:66
  Payload: 'Hello Switch!'
  Updated Switch MAC Table: {'AA:BB:CC:00:11:22': ('127.0.0.1', 58056)}
Client received response: Frame Forwarded Successfully
>>>

```

Ln: 51 Col: 23

4. Using Python's socket library, write a client-server program that models two hosts communicating through a star switch. Demonstrate message forwarding by printing the switch's MAC address table after each frame transmission.

```

File Edit Format Run Options Window Help
class StarSwitch:
    def __init__(self, switch_name="Core-Switch"):
        self.switch_name = switch_name
        self.ports = set()

    def add_port(self, host_id):
        self.ports.add(host_id)
        print(f"[{self.switch_name}] Port added for {host_id}. Active ports: {list(self.ports)}")

    def remove_port(self, host_id):
        if host_id in self.ports:
            self.ports.remove(host_id)
            print(f"[{self.switch_name}] Port removed for {host_id}. Active ports: {list(self.ports)}")
        else:
            print(f"[{self.switch_name}] Host {host_id} not found on any active port")

    def broadcast(self, source_host, frame):
        print(f"[{self.switch_name}] Broadcasting frame from source [{source_host}]")
        if source_host not in self.ports:
            print(f"Warning: Source {source_host} is not connected to the switch")

        delivery_count = 0
        for port_host in self.ports:
            if port_host != source_host:
                print(f"  -> Delivering frame to destination port: {port_host} | Content: '{frame}'")
                delivery_count += 1

        print(f"Broadcast complete. Delivered to {delivery_count} recipient(s).")

# Demonstration of StarSwitch
print("\n--- StarSwitch Class Demonstration ---")
switch = StarSwitch("Enterprise-Switch-01")
switch.add_port("Host-1")
switch.add_port("Host-2")
switch.add_port("Host-3")

# Broadcast a frame from Host-1
switch.broadcast("Host-1", "ARP Request: Who has 192.168.1.5?")

```

Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit AMD64] on win32
Enter "help" below or click "Help" above for more information.

```

>>>
= RESTART: C:\Users\DELL\AppData\Local\Programs\Python\Python313\therla bhail.py

--- StarSwitch Class Demonstration ---

[Enterprise-Switch-01] Port added for Host-1. Active ports: ['Host-1']
[Enterprise-Switch-01] Port added for Host-2. Active ports: ['Host-1', 'Host-2']
[Enterprise-Switch-01] Port added for Host-3. Active ports: ['Host-1', 'Host-2', 'Host-3']

[Enterprise-Switch-01] Broadcasting frame from source [Host-1]...
  -> Delivering frame to destination port: Host-2 | Content: 'ARP Request: Who has 192.168.1.5?'
  -> Delivering frame to destination port: Host-3 | Content: 'ARP Request: Who has 192.168.1.5?'
Broadcast complete. Delivered to 2 recipient(s).
>>>

```

5. Create a class StarSwitch with methods add_port(host_id), remove_port(host_id), and broadcast(frame). Broadcast must send a frame to all ports except the source and log each delivery.

main.py

Visualize

Run

```
1 import random
2
3 def generate_star_topology(num_hosts, switch_name="Switch-Core"):
4     print(f"\n--- Star Topology Diagram ({num_hosts} Hosts & 1 Switch) ---")
5     print(f"Central Switch: {switch_name}\n")
6     connections = []
7     for i in range(1, num_hosts + 1):
8         host_label = f"Host-{i:02d}"
9         cable_length = round(random.uniform(5.0, 95.0), 2)
10        connections.append((host_label, switch_name, cable_length))
11        print(f" {host_label} <--- [{cable_length}m Cat6 UTP] ---> {switch_name}")
12
13    print("\nConnection Summary Table:")
14    print(f"{'Host':<10} | {'Switch':<15} | {'Cable Length':<15}")
15    print("-" * 46)
16    for host, sw, length in connections:
17        print(f"{'host':<10} | {'sw':<15} | {'length':<15} meters")
18
19 generate_star_topology(4)
```

Output

Star Topology Diagram (4 Hosts & 1 Switch) ---

Central Switch: Switch-Core

Host-01 <--- [39.3m Cat6 UTP] ---> Switch-Core

Host-02 <--- [78.18m Cat6 UTP] ---> Switch-Core

Host-03 <--- [62.75m Cat6 UTP] ---> Switch-Core

Host-04 <--- [11.57m Cat6 UTP] ---> Switch-Core

Connection Summary Table:

Host | Switch | Cable Length

Host-01 | Switch-Core | 39.3 meters

Host-02 | Switch-Core | 78.18 meters

Host-03 | Switch-Core | 62.75 meters

Host-04 | Switch-Core | 11.57 meters

=== Code Execution Successful ===